

| Deep Dive: Phase 2 Handout

| Demonstrating the Vulnerable strcpy Operation

C++ Memory Safety Deep Dive Phase 2

Film: intermediate-2-vulnerable-operation.mp4, 7 minutes 20 seconds, 440s total, 1920x1080, 30fps, burned in captions

| Abstract

Phase 1 showed an 8 byte boundary safely inside a 20 byte array capped with `std::min`. Phase 2 removes the cap and performs a real unchecked `strcpy` into a char buffer[8]. The oversized input ABCDEFGHIJKLMNO (15 bytes) writes 16 bytes including the terminating null into 8 bytes, overflowing by 8 bytes into adjacent stack memory. AddressSanitizer detects the overflow as stack buffer overflow and aborts. The program also shows that without sanitizers the same undefined behavior may abort via stack canary or appear to succeed, proving that undefined behavior has no guaranteed outcome. The film is isolated, local and non exploitative.

| How to Use This Handout

- Watch the film once without pausing to follow the story: Phase 1 recall, vulnerable strcpy, overflow, AddressSanitizer report, safety clarification.
- Rewatch Scene 2 for the Phase 1 comparison (20 byte model, 8, 4, 8, HELLO fits versus ABCDEFGHIJKLMNO crosses 7).
- Rewatch Scene 4 for the unchecked copy (first 1234567 fits, then ABCDEFGHIJKLMNO writes 16 into 8, WRITE size 16, [32,40)).
- Use Scene 5 and this handout to read the AddressSanitizer report line by line.
- Use the Compile and Run box below to reproduce both builds on your own machine in an isolated environment with AddressSanitizer.

| Key Concepts

- `strcpy`: C library function `char strcpy(char dest, const char *src)` that copies bytes from `src` to `dest` until the first null. It receives no destination capacity, so it cannot check for overflow.
- AddressSanitizer: Runtime detector built with `fsanitize=address`. It instruments memory accesses and reports invalid reads and writes such as stack buffer overflow with file, line and shadow memory.

- stack buffer overflow: AddressSanitizer error type for a write past the end of a stack buffer (char buffer[8] at [32,40), access at offset 40).
- Buffer: Fixed 8 byte stack array char buffer[8] at line 44. C string capacity is 7 chars plus null.
- Undefined Behavior: What a real out of bounds write is in C++. The standard imposes no requirements. The program may crash, abort via canary, or appear to work.
- Stack Canary: Compiler mitigation fstack protector, default on, that detects stack corruption and aborts with stack smashing detected.
- Input Length versus Capacity: 1234567 is 7 chars plus null equals 8 bytes (fits). ABCDEFGHIJKLMNOP is 15 chars plus null equals 16 bytes (8 past). Length and capacity are separate.
- WRITE of size 16: What AddressSanitizer reports for the oversized copy.
- [32,40) buffer: AddressSanitizer stack frame object for buffer[8] (8 bytes at offset 32, line 44).

Core lesson: strcpy does not know the destination size. Always enforce the boundary explicitly.

| The Vulnerable Code

src/phase2_vulnerable_strcpy.cpp (87 lines, minimal, isolated):

```
constexpr std::size_t BUFFER_SIZE = 8;           // 8 bytes, 7 chars plus
null

void vulnerableFunction(const char *input) {
    char buffer[BUFFER_SIZE];                     // [32,40) buffer line 44
    std::cout << "Buffer capacity: " << BUFFER_SIZE << " bytes\n";
    std::cout << "Input length: " << std::strlen(input) << " bytes\n";
    std::cout << "Calling strcpy(buffer, input)...\n";
    std::strcpy(buffer, input);                   // line 58, SECURITY BUG,
no size check
    std::cout << "Buffer contains: \"" << buffer << "\"\n";
}

int main() {
    vulnerableFunction("1234567");                 // 7 plus null equals 8,
fits, control
    vulnerableFunction("ABCDEFGHJKLMNOP");          // 15 plus null equals 16,
8 past, triggers AddressSanitizer
}
```

Header disclaimers: EDUCATIONAL DEMONSTRATION ONLY, INTENTIONALLY VULNERABLE. Do not use in production. Run only locally with AddressSanitizer.

No exploit payload, no ROP, no return address diagram, no external target, no real credentials.

| The Diagnostic

With `g++ fsanitize=address,undefined g`, the AddressSanitizer report contains:

```
ERROR: AddressSanitizer: stack buffer overflow on address 0x...
WRITE of size 16 at 0x... thread T0
    #0 ... in strcpy
    #1 ... in vulnerableFunction at line 58
    #2 ... in main at line 81

Address is located in stack of thread T0 at offset 40 in frame
    #0 ... in vulnerableFunction at line 43
This frame has 1 object:
    [32, 40) buffer line 44, Access at offset 40 overflows this variable
SUMMARY: AddressSanitizer: stack buffer overflow at line 58 in
vulnerableFunction
Shadow bytes: 00[f3]f3 f3
ABORTING
```

Reading it: type is stack buffer overflow, write is 16 bytes, location is strcpy at line 58 called from main line 81, object is buffer 8 bytes [32,40) line 44, offset is 40 (first byte past), outcome is ABORTING EXIT 1.

Without AddressSanitizer, the same write aborts differently: stack smashing detected terminated EXIT 134 with canary, or EXIT 0 appearing to succeed with fno stack protector. Both are valid undefined behavior.

| Safety Limitations

- Educational only: `phase2_vulnerable` and `phase2_vulnerable_asan` are intentionally vulnerable and must never be deployed, distributed, or run outside an isolated development VM.
- Local and synthetic only: No network, no external host, no privilege escalation, no real user data. Only 1234567 and ABCDEFGHIJKLMNO (printable ASCII).
- AddressSanitizer is a testing aid, not a deployment fix: It detects the bug at runtime during testing. Production code must enforce bounds before the copy.

- No exploit is shown: Film and handout contain no shellcode, no ROP gadget, no address disclosure beyond the local [32,40) range, and no steps to weaponize the overflow.

Every artifact (code header, program banner, film opening and footer, handout, captions) carries: Educational demonstration only, intentionally vulnerable. Run only locally with AddressSanitizer.

| Compile and Run

Verified on g++ 16.2.1, libasan.so.8, Ubuntu 22.04 x86_64.

```
# Without sanitizers (shows stack canary or silent undefined behavior):
g++ -std=c++20 -Wall -Wextra -Wpedantic -g src/phase2_vulnerable_strcpy.cpp
-o phase2_vulnerable
./phase2_vulnerable

# With AddressSanitizer (detects the overflow):
g++ -std=c++20 -Wall -Wextra -Wpedantic -fsanitize=address,undefined -g
src/phase2_vulnerable_strcpy.cpp -o phase2_vulnerable_asan
./phase2_vulnerable_asan

# Tests for data model:
uv run python -m pytest tests -v

# Render video (Pillow and ffmpeg, 1920x1080):
# See animation/phase2_vulnerable_animation.py for render options
# Low quality preview 854x480, high quality 1920x1080
```

| Relationship to Phase 1

Phase 1 Safe Model: One array unsigned char 20, 8 plus 4 plus 8 deterministic. Copy is memcpy with std::min capped. Overflow is none, visualized with LOGICAL BUFFER ENDS HERE marker. Detection is visual FITS and BOUNDARY CROSSED. Exit is EXIT 0. Lesson is where the boundary is.

Phase 2 Real Vulnerability: Real stack char buffer[8] at [32,40). Copy is strcpy uncapped. Overflow is real 16 into 8, adjacent stack bytes overwritten. Detection is stack buffer overflow WRITE size 16 plus canary stack smashing. Exit is EXIT 1 with AddressSanitizer, EXIT 134 with canary, or EXIT 0 silent with fno stack protector. Lesson is what happens when it is actually crossed.

Phase 2 reuses Phase 1 numbers (8, 15, 41 to 4F) so the same boundary learned safely is now crossed for real and detected.

| Next Phase Preview

Phase 3 Apply bounds aware and layered defenses will enforce the boundary:

- Use size aware APIs `string`, `std::array`, `std::span`, `memcpy` with `std::min` .
- Keep AddressSanitizer `fsanitize=address`, undefined in testing.
- Rely on layered mitigations: stack canaries, ASLR, NX/DEP, Control Flow Integrity, static analysis. No single layer is sufficient.
- Takeaway remains: Input length is not buffer capacity. Enforce the boundary every time.

| Files

- `src/phase2_vulnerable_strcpy.cpp`: minimal vulnerable program
- `docs/phase2_asan_analysis.md`: full diagnostic breakdown
- `docs/phase2_specification.md`: specification (sizes, inputs, expected AddressSanitizer output)
- `animation/phase2_video_components.py`: video components (buffer cell, AddressSanitizer panel, disclaimer)
- `docs/phase2_captions.srt`: 35 cues, timed to the 440s film
- `docs/phase2_handout.md`: this handout

| References

- C++ standard `defns.undefined`, `expr.add`: out of bounds access is undefined behavior.
- AddressSanitizer `fsanitize=address` documentation.
- `docs/phase1_handout.md`: Phase 1 safe model background.